

SISTEMA DE CONTROL DE VIÁTICOS

Documentación Técnica del Backend

Arquitectura en capas, servicios, modelo de datos e integraciones

Índice

Índice	2
1. Introducción	4
2. Stack tecnológico.....	4
3. Arquitectura general	4
3.1 Responsabilidad de cada capa	5
4. Estructura de directorios.....	5
4.1 Montaje de rutas	6
4.2 Organización por dominio	6
5. Inventario de rutas y controladores.....	7
5.1 Extremos del dominio de aprobación	7
5.2 Extremos de la revisión final	8
5.3 Extremos de autorización de plazo	9
5.4 Extremos de gastos y facturas.....	9
6. Middlewares.....	9
7. Autenticación y sesión.....	10
8. Modelo de datos	10
8.1 Entidades principales	10
Viaje.....	11
Gasto	11
Gasto_Subitem y Gasto_Tramo_Moneda.....	11
Factura.....	11
Comentario.....	11
Solicitud_Autorizacion_Plazo	11
Correlativo_Recibo	11
8.2 Tratamiento de marcas temporales.....	12
8.3 Índices.....	12
9. Servicios de dominio	12
9.1 Dominio expense.....	13
9.2 Dominio approval	13
9.3 Dominio trip	14
10. Servicios transversales	14
10.1 deadlineService	14
10.2 alcoholDetectionService.....	14
10.3 commentModerationService	14
10.4 emailService	14
10.5 pdfService.....	15
11. Utilidades.....	15

12. Integraciones externas	15
12.1 Extracción de datos de comprobantes.....	15
12.2 Almacenamiento de archivos.....	16
12.3 Correo transaccional	16
13. Flujos de aprobación	16
13.1 Primer flujo: autorización del viaje	16
13.2 Segundo flujo: rendición de gastos	16
13.3 Rechazo y ciclos de revisión	17
13.4 Control de concurrencia	17
13.5 Alertas de revisión.....	17
14. Gestión de plazos	17
15. Convenciones de codificación	18
16. Scripts de base de datos.....	18

1. Introducción

Este documento describe la arquitectura del servidor de aplicaciones del Sistema de Control de Viáticos (SCV). El backend expone una API REST que centraliza las reglas de negocio del sistema, la persistencia de datos, el control de acceso por rol y las integraciones con servicios externos.

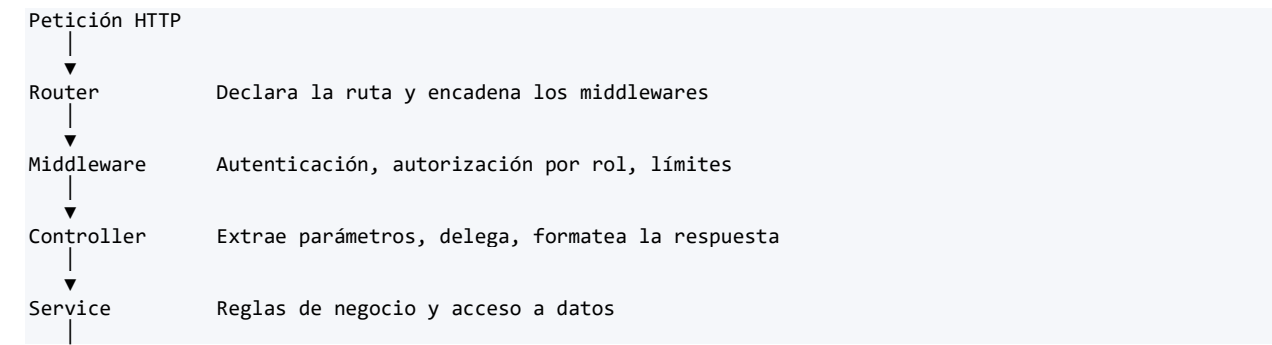
El sistema gestiona dos flujos de aprobación consecutivos e independientes. El primero autoriza la realización del viaje y la asignación de fondos; el segundo valida la rendición de los gastos efectivamente incurridos. Cada flujo involucra distintos perfiles de usuario y transiciones de estado propias.

2. Stack tecnológico

Tecnología	Propósito
Node.js	Entorno de ejecución del servidor.
Express	Framework de enrutamiento y middlewares HTTP.
Supabase (PostgreSQL)	Base de datos relacional y almacenamiento de archivos.
jsonwebtoken	Emisión y verificación de tokens de sesión.
bcrypt	Cifrado unidireccional de contraseñas.
Google Generative AI	Extracción de datos de comprobantes y moderación de contenido.
Brevo (sib-api-v3-sdk)	Envío de correo electrónico transaccional.
Multer	Procesamiento de archivos recibidos en formularios multiparte.
Jim	Lectura y manipulación de imágenes en memoria.
qrcode-reader	Decodificación de códigos QR presentes en las facturas.
Cheerio	Análisis del documento HTML devuelto por el portal del SIAT.
Axios	Cliente HTTP para las consultas a servicios externos.
Helmet	Cabeceras de seguridad HTTP.
cookie-parser	Lectura de cookies en las peticiones entrantes.
CORS	Restricción de los orígenes autorizados.

3. Arquitectura general

El servidor sigue una arquitectura en capas con responsabilidades estrictamente delimitadas. Cada petición atraviesa la misma secuencia de componentes, lo que hace predecible tanto el flujo de ejecución como la ubicación de cualquier pieza de lógica.



3.1 Responsabilidad de cada capa

Capa	Responsabilidad	Restricción
Router	Asociar una ruta y verbo HTTP con su controlador, declarando los middlewares aplicables.	No contiene lógica.
Middleware	Validar la sesión, el rol y las restricciones de acceso previas al controlador.	No accede a reglas de negocio.
Controller	Leer parámetros y cuerpo, invocar el servicio y traducir el resultado a una respuesta HTTP.	No consulta la base de datos.
Service	Ejecutar validaciones de negocio, consultar y modificar datos, orquestar servicios transversales.	No conoce req ni res.

Esta separación se refleja en la firma de las funciones: un controlador siempre recibe los objetos de petición y respuesta, mientras que un servicio recibe parámetros explícitos y devuelve un objeto plano. Cuando un servicio detecta un error de negocio, retorna un objeto con las propiedades error y status, que el controlador traduce al código HTTP correspondiente.

```
const takeExpenseReview = async (req, res) => {
  const {tripId} = req.params
  const supervisorId = req.user.id_usuario
  const result = await reviewService.takeExpenseReview(tripId, supervisorId)
  if (result.error) {
    return res.status(result.status).json({error: result.error})
  }
  else {
    return res.json({message: result.message})
  }
};
```

4. Estructura de directorios

```
src/
├── config/
│   └── supabase.js           Cliente de conexión
├── routes/
│   ├── admin/admin.js
│   ├── approval/
│   │   ├── review.js
│   │   ├── reviewer.js
│   │   ├── approver.js
│   │   ├── treasurer.js
│   │   └── deadlineAuthorization.js
│   ├── catalog/
│   │   ├── role.js
│   │   ├── position.js
│   │   ├── expenseCategory.js
│   │   ├── tax.js
│   │   └── audit.js
│   ├── expense/
│   │   ├── expense.js
│   │   ├── invoice.js
│   │   ├── invoiceDetail.js
│   │   ├── image.js
│   │   ├── supplier.js
│   │   └── comment.js
│   ├── trip/trip.js
│   └── user/
│       ├── user.js
│       └── auth.js
├── controllers/              Misma división que routes
├── services/
└── admin/
```



4.1 Montaje de rutas

El archivo app.js registra la configuración global, que incluye cabeceras de seguridad, política de orígenes, análisis del cuerpo de la petición y lectura de cookies, montando cada router bajo su prefijo correspondiente.

```
app.use('/role', roleRoute)
app.use('/position', positionRoute)
app.use('/audit', auditRoute)
app.use('/comment', commentRoute)
app.use('/expense-category', expenseCategoryRoute)
app.use('/invoice-detail', invoiceDetailRoute)
app.use('/invoice', invoiceRoute)
app.use('/expense', expenseRoute)
app.use('/image', imageRoute)
app.use('/tax', taxRoute)
app.use('/review', reviewRoute)
app.use('/supplier', supplierRoute)
app.use('/user', userRoute)
app.use('/trip', tripRoute)
app.use('/treasurer', treasurerRoute)
app.use('/auth', authRoute)
app.use('/admin', adminRoute)
app.use('/reviewer', reviewerRoute)
app.use('/deadline-authorization', deadlineAuthorizationRoute)
app.use('/approver', approverRoute)
```

4.2 Organización por dominio

Los directorios routes, controllers y services replican la misma división, de modo que cada funcionalidad puede seguirse verticalmente a través de las tres capas.

Dominio	Alcance
user	Autenticación, gestión de usuarios, perfil y recuperación de contraseña.
trip	Ciclo de vida del viaje y sus comentarios asociados.
expense	Gastos, facturas, proveedores, imágenes, detalle y recibos.
approval	Los dos flujos de aprobación con sus perfiles intervinientes.
catalog	Entidades maestras: roles, cargos, categorías, impuestos y auditoría.
admin	Panel administrativo y métricas agregadas.

5. Inventario de rutas y controladores

Cada archivo de rutas declara los extremos de su dominio y los asocia a las funciones del controlador homónimo. La tabla siguiente enumera la totalidad de los archivos de la capa de enrutamiento con su prefijo de montaje.

Archivo de rutas	Prefijo	Controlador asociado
catalog/role.js	/role	roleController
catalog/position.js	/position	positionController
catalog/expenseCategory.js	/expense-category	expenseCategoryController
catalog/tax.js	/tax	taxController
catalog/audit.js	/audit	auditController
user/auth.js	/auth	authController
user/user.js	/user	userController
trip/trip.js	/trip	tripController
expense/expense.js	/expense	expenseController
expense/invoice.js	/invoice	invoiceController
expense/invoiceDetail.js	/invoice-detail	invoiceDetailController
expense/image.js	/image	imageController
expense/supplier.js	/supplier	supplierController
expense/comment.js	/comment	commentController
approval/review.js	/review	reviewController
approval/reviewer.js	/reviewer	reviewerController
approval/approver.js	/approver	approverController
approval/treasurer.js	/treasurer	treasurerController
approval/deadlineAuthorization.js	/deadline-authorization	deadlineAuthorizationController
admin/admin.js	/admin	adminController

5.1 Extremos del dominio de aprobación

El archivo `review.js` concentra los dos flujos del supervisor, diferenciados por el segmento inicial de la ruta: `trip-review` para la aprobación previa y `expense-review` para la rendición. Ambos conjuntos replican la misma estructura de operaciones.

Método y ruta	Operación
GET <code>/trip-review/pending</code>	Viajes pendientes de aprobación previa.
GET <code>/trip-review/mine</code>	Viajes asignados al supervisor.
POST <code>/trip-review/:tripId/take</code>	Asignación del viaje al supervisor.
POST <code>/trip-review/:tripId/return</code>	Devolución del viaje a la bandeja general.
GET <code>/trip-review/:tripId</code>	Detalle del viaje con sus comentarios.
POST <code>/trip-review/:tripId/approve</code>	Aprobación de la solicitud de viaje.
POST <code>/trip-review/:tripId/reject</code>	Rechazo de la solicitud de viaje.
POST <code>/trip-review/:tripId/comment</code>	Alta de observación.
PUT <code>/trip-review/:tripId/comment/:commentId</code>	Edición de observación.
DELETE <code>/trip-review/:tripId/comment/:commentId</code>	Baja de observación.
GET <code>/expense-review/pending</code>	Rendiciones pendientes de revisión.
GET <code>/expense-review/mine</code>	Rendiciones asignadas al supervisor.
POST <code>/expense-review/:tripId/take</code>	Asignación de la rendición.
POST <code>/expense-review/:tripId/return</code>	Devolución de la rendición.
GET <code>/expense-review/:tripId</code>	Detalle con gastos, totales y alertas.
POST <code>/expense-review/:tripId/approve</code>	Aprobación de la rendición.
POST <code>/expense-review/:tripId/reject</code>	Rechazo de la rendición.
POST <code>/expense-review/:tripId/comment</code>	Observación sobre un gasto puntual.
PUT <code>/expense-review/:tripId/comment/:commentId</code>	Edición de la observación.
DELETE <code>/expense-review/:tripId/comment/:commentId</code>	Baja de la observación.

Todos los extremos del archivo exigen sesión válida y rol de supervisor mediante la combinación de los middlewares de autenticación y autorización.

5.2 Extremos de la revisión final

Método y ruta	Operación
GET <code>/reviewer/pending</code>	Rendiciones pendientes de revisión final.
GET <code>/reviewer/mine</code>	Historial de revisiones resueltas.
GET <code>/reviewer/:tripId</code>	Detalle de la rendición con totales y alertas.
POST <code>/reviewer/:tripId/approve</code>	Aprobación definitiva.
POST <code>/reviewer/:tripId/reject</code>	Rechazo definitivo.

Método y ruta	Operación
POST /reviewer/:tripId/comment	Alta de observación sobre un gasto.
PUT /reviewer/:tripId/comment/:commentId	Edición de observación.
DELETE /reviewer/:tripId/comment/:commentId	Baja de observación.

5.3 Extremos de autorización de plazo

Método y ruta	Operación
POST /deadline-authorization/trip/:tripId/request	Solicitud de extensión por parte del empleado.
GET /deadline-authorization/trip/:tripId/status	Estado de la última solicitud del viaje.
GET /deadline-authorization/pending	Solicitudes pendientes de resolución.
GET /deadline-authorization/history	Solicitudes ya resueltas.
POST /deadline-authorization/:requestId/approve	Aprobación de la solicitud.
POST /deadline-authorization/:requestId/reject	Rechazo con observación obligatoria.

5.4 Extremos de gastos y facturas

Método y ruta	Operación
GET /expense/:expenseId	Detalle completo del gasto con sus relaciones.
POST /expense	Registro de un gasto con su comprobante.
PUT /expense/:expenseId	Actualización del gasto y sus subelementos.
DELETE /expense/:expenseId	Eliminación del gasto y registros dependientes.
POST /expense/:expenseId/receipt	Emisión y envío del recibo individual.
POST /expense/trip/:tripId/receipt/:type	Emisión del recibo agrupado por tipo.
POST /invoice/extract	Extracción de datos desde la imagen del comprobante.
POST /invoice	Registro de la factura con su detalle.
PUT /invoice/:expenseId	Actualización de la factura registrada.

Los extremos que reciben archivos emplean Multer configurado con almacenamiento en memoria, de modo que el contenido queda disponible como búfer para su procesamiento y posterior carga al almacenamiento remoto.

6. Middlewares

Middleware	Función
auth	Verifica el token de sesión y adjunta los datos del usuario a la petición.
roleAuth	Recibe la lista de roles habilitados y rechaza al solicitante no autorizado.

Middleware	Función
loginRateLimiter	Limita los intentos fallidos de autenticación por origen.
treasurerPosition	Restringe operaciones de tesorería según el cargo del usuario.

El middleware `roleAuth` se declara en cada ruta indicando explícitamente los perfiles habilitados, lo que hace visible la restricción sin necesidad de inspeccionar el controlador.

```
router.post('/expense-review/:tripId/take',
  authMiddleware,
  roleMiddleware(['SUPERVISOR']),
  reviewController.takeExpenseReview)
```

La existencia de `treasurerPosition` responde a que la función de tesorería no constituye un rol del sistema sino un cargo dentro de la organización, por lo que su verificación se realiza contra el puesto asignado al usuario y no contra el rol contenido en el token.

7. Autenticación y sesión

La autenticación se resuelve mediante JSON Web Tokens. Al iniciar sesión, el servidor verifica la contraseña contra su hash almacenado mediante `bcrypt` y emite un token que incorpora el identificador del usuario y su rol.

- Las contraseñas nunca se almacenan en claro; se guarda únicamente su hash.
- El campo `ultima_cambio_contrasenia` permite exigir la renovación periódica de credenciales.
- El campo `refresh_token_invalido_desde` invalida las sesiones emitidas antes de un cambio de contraseña.
- La tabla `Codigo_Verificacion` almacena los códigos temporales de recuperación con su fecha de expiración.
- La tabla `Auditoria` registra los eventos de ingreso, salida y cambio de clave.

8. Modelo de datos

El esquema comprende diecinueve tablas organizadas en agrupaciones funcionales.

Agrupación	Tablas
Catálogo	Rol, Cargo, Categoria_Gasto, Impuesto, Proveedor
Usuarios	Usuario, Codigo_Verificacion, Auditoria
Viajes	Viaje, Solicitud_Autorizacion_Plazo
Gastos	Gasto, Gasto_Subitem, Gasto_Tramo_Moneda, Imagen
Facturas	Factura, Detalle_Factura, Factura_Impuestos
Transversal	Comentario, Correlativo_Recibo

8.1 Entidades principales

Viaje

Entidad central del sistema, su columna estado gobierna ambos flujos de aprobación mediante una restricción de verificación que enumera los nueve estados válidos. Mantiene además las referencias a los usuarios asignados en cada instancia de revisión, que son supervisor, aprobador, revisor y tesorero, junto con el contador ciclo_revision, que permite distinguir las observaciones de la ronda vigente respecto de las anteriores.

La bandera fue_iniciado distingue un viaje rechazado durante la aprobación previa de uno rechazado durante la rendición, dado que ambos comparten el mismo valor de estado. La columna tiene_alcohol almacena el resultado del análisis de los productos facturados.

Gasto

Registra cada erogación. Almacena tanto el importe como los valores de retención impositiva calculados en el momento del registro: base imponible, RC-IVA, IUE, IT e importe costo. Persistir estos valores en lugar de recalcularlos garantiza que los reportes históricos permanezcan consistentes ante eventuales cambios en las alícuotas.

Los campos moneda, tipo_cambio y monto_moneda_origen registran la conversión aplicada, mientras que es_gasto_internacional determina la moneda de referencia del importe.

Gasto_Subitem y Gasto_Tramo_Moneda

Un gasto puede desglosarse en subítems, cada uno con su descripción e importe; el monto total pasa entonces a calcularse como su sumatoria. Los gastos internacionales pueden además registrar múltiples tramos de conversión, cada uno con moneda de origen, importe, tipo de cambio y equivalente en dólares.

Factura

Vinculada a un gasto mediante una relación uno a uno. Almacena el número de comprobante, la fecha de emisión y el monto parcial anterior a impuestos. Una restricción de unicidad sobre número, fecha y proveedor evita el registro duplicado del mismo comprobante. Su detalle se compone de los productos facturados, y la tabla Factura_Impuestos vincula la factura con los impuestos aplicados.

Comentario

Unifica las observaciones emitidas durante las revisiones y las justificaciones de reembolso, distinguidas por la columna tipo. Se vincula siempre a un viaje y opcionalmente a un gasto específico, lo que permite observar una erogación individual dentro de una rendición. El campo ciclo_revision asocia cada observación a la ronda en que fue emitida.

Solicitud_Autorizacion_Plazo

Registra las solicitudes de extensión del plazo de carga. Contiene el motivo expuesto por el empleado, el estado de la solicitud, el revisor que la resolvió, su observación en caso de rechazo y las marcas temporales de solicitud y respuesta. Esta última determina la fecha desde la cual se computa la extensión concedida.

Correlativo_Recibo

Tabla de una sola fila que mantiene el contador de recibos emitidos. Su incremento se realiza mediante una función almacenada que actualiza y devuelve el nuevo valor en una única operación, evitando que dos emisiones simultáneas obtengan el mismo número.

8.2 Tratamiento de marcas temporales

Todas las columnas de fecha y hora se declaran como `timestampz`. Esta decisión resolvió un defecto identificado durante el desarrollo: al utilizar el tipo `timestamp` sin zona horaria, PostgreSQL descartaba la información de uso al almacenar el valor, y el cliente lo interpretaba posteriormente como hora local, produciendo un desplazamiento de cuatro horas respecto del horario de Bolivia.

```
ALTER TABLE "Comentario"  
ALTER COLUMN fecha TYPE timestampz  
USING fecha AT TIME ZONE 'UTC';
```

Complementariamente, los servicios que comparan fechas calendario incorporan funciones auxiliares que convierten explícitamente a la zona horaria local antes de operar, evitando que una acción realizada en horario nocturno se compute como perteneciente al día siguiente.

```
const boliviaOffsetHours = -4  
  
const getBoliviaToday = () => {  
  const now = new Date()  
  const boliviaTime = new Date(now.getTime() + boliviaOffsetHours * 60 * 60 * 1000)  
  return boliviaTime.toISOString().split('T')[0]  
};
```

8.3 Índices

Se definieron índices sobre las columnas empleadas con mayor frecuencia como criterio de filtrado o unión.

Índice	Columna
idx_viaje_id_usuario	Viaje(id_usuario)
idx_viaje_estado	Viaje(estado)
idx_viaje_supervisor	Viaje(id_supervisor_asignado)
idx_gasto_id_viaje	Gasto(id_viaje)
idx_gasto_subitem_id_gasto	Gasto_Subitem(id_gasto)
idx_gasto_tramo_id_gasto	Gasto_Tramo_Moneda(id_gasto)
idx_comentario_id_viaje	Comentario(id_viaje)
idx_comentario_id_gasto	Comentario(id_gasto)
idx_factura_id_gasto	Factura(id_gasto)
idx_detalle_factura_id_factura	Detalle_Factura(id_factura)
idx_solicitud_id_viaje	Solicitud_Autorizacion_Plazo(id_viaje)
idx_solicitud_estado	Solicitud_Autorizacion_Plazo(estado)
idx_auditoria_id_usuario	Auditoria(id_usuario)
idx_codigo_verificacion_id_usuario	Codigo_Verificacion(id_usuario)

9. Servicios de dominio

9.1 Dominio expense

Servicio	Responsabilidad
expenseService	Alta, edición y baja de gastos; cálculo de retenciones; gestión de subítems y tramos.
invoiceService	Registro y actualización de facturas con su detalle e impuestos asociados.
invoiceExtractionService	Obtención de los datos del comprobante a partir de la imagen.
receiptService	Generación y envío de recibos individuales y agrupados.
supplierService	Búsqueda o creación del proveedor a partir de su denominación.

La función de cálculo de retenciones aplica una fórmula distinta según el tipo de comprobante: los gastos con factura o recibo no generan retención; las compras sin factura aplican IUE del cinco por ciento e IT del tres por ciento sobre una base incrementada; los servicios sin factura aplican RC-IVA del trece por ciento e IT del tres por ciento. Los gastos internacionales quedan exentos de todo cálculo de retención.

```
const calculateRetentions = (amount, type, isInternational) => {
  const amountNum = parseFloat(amount)
  if (isInternational || type === 'F' || type === 'R') {
    return {base_imponible: amountNum, retencion_rc_iva: 0,
      retencion_iue: 0, retencion_it: 0, importe_costo: amountNum}
  }
  if (type === 'C') {
    const base = parseFloat((amountNum / 0.92).toFixed(2))
    const iue = parseFloat((base * 0.05).toFixed(2))
    const it = parseFloat((base * 0.03).toFixed(2))
    return {base_imponible: base, retencion_rc_iva: 0,
      retencion_iue: iue, retencion_it: it, importe_costo: base}
  }
  ...
};
```

9.2 Dominio approval

Servicio	Responsabilidad
reviewService	Ambos flujos del supervisor: bandejas, asignación, devolución, aprobación, rechazo y comentarios.
reviewerService	Revisión final: bandejas, detalle, aprobación definitiva, rechazo y comentarios.
approverService	Aprobación del viaje en el primer flujo y emisión del memorándum.
treasurerService	Validación y edición de los montos asignados al viaje.
deadlineAuthorizationService	Solicitud, consulta, aprobación y rechazo de extensiones de plazo.
expenseSummaryService	Cálculo de acumulados, excesos de presupuesto y alertas de revisión.

El servicio `expenseSummaryService` centraliza el cómputo que consumen tanto el supervisor como el revisor, garantizando que ambos perfiles observen cifras idénticas. Separa los gastos por moneda, acumula sus importes, contrasta el resultado contra el presupuesto asignado y construye el conjunto de alertas.

```
const calculateExpenseSummary = (expenses, trip) => {
  const nationalExpenses = (expenses || []).
    .filter((expense) => !expense.es_gasto_internacional)
  const internationalExpenses = (expenses || []).
    .filter((expense) => !!expense.es_gasto_internacional)
  const accumulatedExpense = nationalExpenses
    .reduce((sum, e) => sum + parseFloat(e.monto_total || 0), 0)
```

```
...  
return {accumulatedExpense, accumulatedExpenseUsd,  
        exceedsBudget, exceedsBudgetUsd}  
};
```

9.3 Dominio trip

Servicio	Responsabilidad
tripService	Alta, edición, listados, detalle y transiciones de estado del viaje.
tripCommentService	Alta, edición y baja de comentarios, con moderación previa del contenido.

La totalidad de las operaciones sobre comentarios, tanto las del primer flujo como las de la rendición, se delegan en tripCommentService, que aplica de forma centralizada las validaciones de longitud, la verificación de pertenencia del gasto al viaje, la moderación del contenido y la asignación del ciclo de revisión vigente.

10. Servicios transversales

10.1 deadlineService

Determina si un gasto puede registrarse en función de dos criterios independientes: que la fecha del gasto se encuentre dentro del período del viaje, y que la carga se realice dentro del plazo de tolerancia posterior a su finalización, establecido en cuatro días.

Cuando el plazo ha vencido, verifica la existencia de una autorización de extensión aprobada, computando su vigencia a partir de la fecha de respuesta convertida a la zona horaria local. La validación de la fecha del gasto contra el período del viaje se aplica siempre, independientemente de la extensión: esta amplía el margen para cargar, no el rango de fechas admisibles.

10.2 alcoholDetectionService

Determina si una rendición incluye bebidas alcohólicas, condición que genera una alerta para los revisores. Opera en dos etapas: primero contrasta los productos contra un catálogo de términos conocidos, que incluye denominaciones locales, marcas comerciales y establecimientos; si el resultado es ambiguo, consulta al modelo de lenguaje.

Incorpora además un listado de términos claramente inocuos que permite descartar productos sin necesidad de invocar el modelo, reduciendo el número de consultas externas. El servicio expone también la validación de contenido inapropiado empleada en la moderación de comentarios.

10.3 commentModerationService

Verifica que el texto de un comentario no contenga términos prohibidos, contrastándolo contra el catálogo definido en utils/forbiddenWords. La validación se ejecuta antes de persistir el comentario, y su rechazo se comunica mediante un mensaje que el cliente presenta dentro del modal correspondiente.

10.4 emailService

Centraliza el envío de correo transaccional a través de Brevo. Expone tres operaciones: el envío del código de verificación, la validación de existencia de una dirección mediante un servicio externo de reputación, y el envío genérico con adjuntos opcionales.

Todas las notificaciones del sistema, que incluyen códigos de verificación, solicitudes de autorización, resultados de aprobación y recibos, se emiten mediante este servicio, empleando una plantilla común que mantiene la identidad visual institucional mediante un encabezado con línea de acento, bloques de datos destacados y un pie explicativo.

10.5 pdfService

Genera los documentos oficiales del sistema. Se apoya en el utilitario numberToWords para expresar los importes en letras, requisito habitual en la documentación contable, y en la tabla Correlativo_Recibo para asignar numeración secuencial a los recibos emitidos.

11. Utilidades

Archivo	Contenido
forbiddenWords	Catálogo de términos vedados en los comentarios del sistema.
numberToWords	Conversión de importes numéricos a su expresión en letras.
textNormalizer	Normalización de cadenas para comparaciones sin acentos ni mayúsculas.

12. Integraciones externas

12.1 Extracción de datos de comprobantes

El servicio invoiceExtractionService implementa una estrategia de resolución en cascada, ordenada de mayor a menor confiabilidad de la fuente:

- Lectura del código QR presente en la imagen mediante Jimp y qrcode-reader.
- Si el código apunta al portal del SIAT, consulta directa a la autoridad tributaria y análisis del HTML devuelto mediante Cheerio. Esta ruta produce datos verificados oficialmente, incluyendo el detalle de productos.
- Si el código responde a un formato genérico, se interpretan sus parámetros para extraer NIT, número de factura, fecha e importe.
- Como última instancia, la imagen se envía al modelo generativo, que devuelve los campos estructurados en formato JSON.

```
const extractInvoiceData = async (file) => {
  const qrText = await readQrFromBuffer(file.buffer)
  if (qrText) {
    let qrData = null
    if (qrText.includes('siat.impuestos.gob.bo')) {
      qrData = await getDataFromSiatQr(qrText)
    }
    if (!qrData) {
      qrData = parseGenericQr(qrText)
    }
    if (qrData) {
      return qrData
    }
  }
}
```

```

    return await extractInvoiceWithGemini(file.buffer, file.mimetype)
}

```

La invocación al modelo incorpora reintentos con espera incremental, dado que las respuestas de un modelo generativo pueden ocasionalmente no ajustarse al formato solicitado. El indicador `extraido_por_qr` acompaña al resultado, permitiendo al cliente señalar visualmente que los datos provienen de una fuente verificada.

La instrucción enviada al modelo especifica el contexto tributario boliviano: aclara que el término «Importe» corresponde al total con impuesto incluido, que la alícuota del IVA es del trece por ciento, y que el campo de impuesto debe expresarse como monto y no como porcentaje.

12.2 Almacenamiento de archivos

Las imágenes de comprobantes y las fotografías de perfil se alojan en Supabase Storage. La base de datos conserva únicamente la URL pública del archivo, evitando almacenar contenido binario en las tablas. El nombre del archivo incorpora el identificador del gasto y una marca temporal, previniendo colisiones.

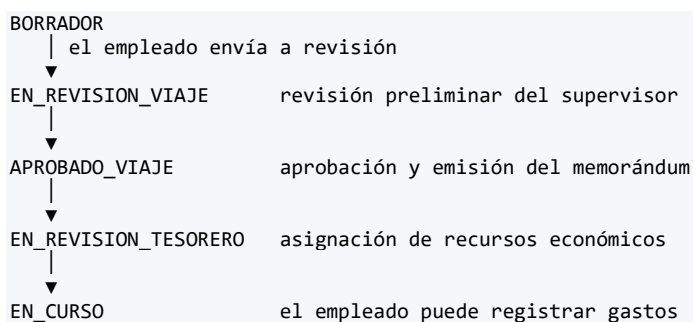
12.3 Correo transaccional

El envío se realiza a través de la API de Brevo. La validación previa de las direcciones se apoya en un servicio de reputación externo que verifica el formato, la existencia del dominio y la capacidad de recepción; ante un fallo de ese servicio, la validación se considera superada para no bloquear el registro de usuarios por una causa ajena al sistema.

13. Flujos de aprobación

13.1 Primer flujo: autorización del viaje

Se inicia cuando el empleado envía a revisión un viaje en estado borrador y concluye con la asignación efectiva de los fondos.

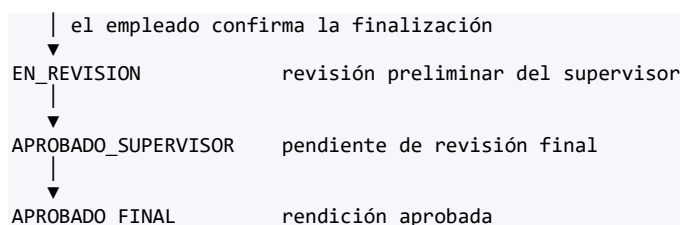


El memorándum emitido en la instancia del aprobador detalla el motivo, el destino, las fechas y el medio de transporte autorizado, sin incluir el importe asignado, dado que su determinación corresponde a la instancia posterior de tesorería.

13.2 Segundo flujo: rendición de gastos

Comienza cuando el empleado da por concluida la carga de sus gastos y culmina con la aprobación definitiva de la rendición.

EN_CURSO



13.3 Rechazo y ciclos de revisión

En ambos flujos, el rechazo conduce al estado **RECHAZADO** e incrementa el contador `ciclo_revision`, habilitando al empleado a corregir y reenviar. Las observaciones de ciclos anteriores se conservan como historial, pero solo las del ciclo vigente se consideran para validar un nuevo rechazo.

El sistema exige que exista al menos una observación registrada en el ciclo actual antes de permitir el rechazo, evitando devoluciones sin fundamento. Esta validación se replica en el cliente para anticipar el aviso al usuario antes de enviar la petición.

13.4 Control de concurrencia

Dado que varios supervisores pueden acceder simultáneamente a la bandeja de revisiones pendientes, la asignación de un viaje verifica que el campo de supervisor asignado se encuentre vacío antes de establecerlo. El segundo intento sobre el mismo viaje recibe un error específico que el cliente traduce en un aviso mediante un modal dedicado.

13.5 Alertas de revisión

El servicio `expenseSummaryService` construye el conjunto de alertas que acompaña a cada rendición y determina su estado de conformidad.

Alerta	Condición
EXCESO_PRESUPUESTO	El acumulado supera el monto asignado en cualquiera de las dos monedas.
ALCOHOL	El análisis de los productos facturados detectó bebidas alcohólicas.

La presencia de al menos una alerta determina que el estado de revisión sea **OBSERVADO**; su ausencia, **CONFORME**. Este valor alimenta las pestañas de filtrado de las bandejas de entrada.

14. Gestión de plazos

El plazo de tolerancia estándar concede cuatro días posteriores a la finalización del viaje para completar la carga de gastos. Vencido ese margen, el registro queda bloqueado y el empleado debe solicitar una autorización al revisor.

Estado	Efecto
Sin solicitud	El registro queda bloqueado; se habilita el botón de solicitud.
PENDIENTE	El registro permanece bloqueado hasta la resolución del revisor.
APROBADA vigente	El registro se habilita durante cuatro días desde la fecha de respuesta.
APROBADA vencida	El registro se bloquea nuevamente; puede solicitarse otra extensión.

Estado	Efecto
RECHAZADA	El registro permanece bloqueado; puede formularse una nueva solicitud.

La consulta de estado devuelve, además del registro de la solicitud, dos campos calculados: `extension_vencida`, que indica si la vigencia expiró, y `limite_extendido`, con la fecha hasta la cual se autoriza el registro.

15. Convenciones de codificación

- El código se escribe en inglés; los mensajes dirigidos al usuario, en español.
- Cada función lleva un comentario de una línea que describe su propósito.
- No se emplean operadores ternarios en la lógica de control.
- Las palabras clave `else` y `catch` se ubican en línea nueva tras el cierre de la llave.
- Los literales de objeto se escriben sin espacios internos.
- Los servicios devuelven objetos planos; no lanzan excepciones para errores de negocio previstos.
- Las operaciones auxiliares que no deben interrumpir el flujo principal —notificaciones, recálculos— se ejecutan dentro de bloques de captura que registran la incidencia sin propagarla.

16. Scripts de base de datos

El directorio `database` contiene los scripts necesarios para reconstruir el esquema desde cero, separados por responsabilidad.

Archivo	Contenido
<code>schema.sql</code>	Definición de las diecinueve tablas, restricciones de verificación, claves foráneas e índices.
<code>seeds.sql</code>	Datos iniciales: roles, cargos, categorías de gasto con su cuenta contable e impuestos.
<code>functions.sql</code>	Función de incremento atómico del correlativo de recibos.
<code>rls.sql</code>	Activación de seguridad a nivel de fila en todas las tablas.

La activación de RLS responde a que el backend accede a la base de datos mediante la clave de servicio, que omite estas políticas. Habilitarlas garantiza que ninguna conexión externa pueda leer o modificar datos directamente, dejando al servidor de aplicaciones como único punto de acceso.

El script de datos iniciales incorpora las categorías de gasto con el formato que espera el sistema: el código de la cuenta contable seguido de su denominación, dado que la exportación de la planilla extrae ese código del nombre de la categoría.